Keywords: tool calling • LLM agents • programmatic invocation • benchmark evaluation

The Bitter Lesson of Tool Calling

Programmatic Python-Stub Invocation Outperforms JSON Tool Calling in 11 of 14 Models Across a Generation-Spanning Benchmark

By Ishan Patel, Sahil Sen, Elias Lumer, Vamse Kumar Subbiah

arXiv:2608.06370 • Preprint, August 2026

The dominant paradigm for giving language models access to external tools—native JSON tool calling—was adopted by convention, not by systematic study. A new paper comparing Programmatic Tool Calling (PTC) against JSON tool calling across 14 models on BFCL v4 finds that PTC matches or beats JSON in 11 of 14 models, with the GPT-5.6 family gaining 10.6% and PTC winning 13 of 14 models under parallel fan-out tasks.

AT A GLANCE

Problem: The JSON tool-calling format is the default for LLM agents, but has never been systematically compared to programmatic alternatives at scale.

Why it matters: Tool calling underlies virtually every production LLM agent; a 10% accuracy gap from a format choice is a free performance gain.

Method: Evaluate PTC (typed Python stubs + code execution) vs. JSON across 14 models on BFCL v4 under three tracks: standard, parallel fan-out, context rot.

Result: PTC wins in 11/14 standard, 13/14 parallel fan-out; JSON drops 2.3% under context rot while PTC holds steady.

Evidence: GPT-5.6 family gains +10.6% with PTC; results replicated across model families and generations.

The Big Picture

Every production LLM agent system must make a choice the field has rarely examined: how should tools be presented to the language model? The two dominant options are **native JSON tool calling**, where the model emits a structured JSON object naming the tool and providing argument values, and **Programmatic Tool Calling (PTC)**, where tools are defined as typed Python function stubs and the model writes executable Python code to invoke them.

JSON tool calling became the standard through historical momentum—it was the format exposed by early commercial APIs, and the ecosystem solidified around it. This paper asks whether that default is actually optimal, and finds systematically that it is not. PTC exploits models' extensive code training, allows the model to reason about function signatures before committing to a call, and composes naturally with Python's type system for argument validation.

Why Existing Approaches Fall Short

JSON tool calling has two structural weaknesses that PTC avoids. First, the JSON schema for a tool is typically serialized as a string in the prompt, forcing the model to parse an ad hoc format rather than the typed function signatures it encountered during code pretraining. Second, under parallel fan-out—where multiple tools must be invoked in a single turn—JSON requires emitting a list of JSON objects, a format the model sees rarely, whereas PTC naturally supports multi-function Python calls with loop and control-flow structures.

Under **context rot**—the realistic condition where prior conversation turns accumulate in the context window, cluttering it with tool definitions and results from earlier steps—JSON performance degrades 2.3% on average while PTC holds steady. The hypothesis is that Python code provides stronger structural anchoring than JSON blobs, making it more robust to noisy context.

Core Mechanism

In PTC, each tool is presented as a typed Python stub:

```
def search_web(query: str,
               max_results: int = 5
               ) -> list[dict]:
    """Search the web for query."""
    ...
```

The model is instructed to write Python code invoking these stubs, with actual execution handled by the agent harness. Results are returned as Python objects and made available to the next step. This mirrors the model's training distribution for code: function calls, not JSON blobs.

Technical Formulation

Let $\mathcal{T} = \{t_1, \ldots, t_K\}$ be the set of available tools and q the user query. Under JSON calling, the model produces:

$$a_{\text{JSON}} = \{\text{“name”} : t_k,\ \text{“args”} : \{\ldots\}\}$$

Under PTC, the model produces executable Python:

$$a_{\text{PTC}} = t_k(\text{arg}_1 = v_1, \ldots, \text{arg}_m = v_m)$$

Functional correctness is measured on BFCL v4 by checking that a matches the reference invocation: $\mathbb{1}[a = a^*]$.

The aggregate metric is accuracy averaged over all items in each evaluation track.

The context-rot track injects n prior turns of tool-use history into the context before the test query:

$$\Delta_{\text{rot}} = \text{Acc}(n = 0) - \text{Acc}(n = N_{\text{rot}})$$

with $\Delta_{\text{rot}}^{\text{PTC}} \approx 0$ and $\Delta_{\text{rot}}^{\text{JSON}} \approx -2.3\%$ on average.

Learning or Inference Procedure

No training. Both PTC and JSON are evaluated at inference time by changing only the prompt format and tool-presentation schema. No fine-tuning is performed. The study is therefore a pure format comparison, isolating the effect of tool presentation from any model-level differences.

BFCL v4 provides three tracks:

- **Standard:** single-turn, well-scoped tool calls
- **Parallel fan-out:** multiple tools invoked in one turn
- **Context rot:** accumulated prior turns in context

All 14 models are evaluated identically under both formats.

What the Guarantee Says

No formal guarantee is provided. The empirical claim is that PTC's advantage is consistent across 14 models spanning multiple families and training generations, making it unlikely to be an artifact of any single model's quirks. The parallel fan-out win rate of 13/14 is particularly robust. The paper does not claim PTC universally dominates, but establishes a strong empirical prior for practitioners choosing a tool-calling format.

Experimental Findings

- **Standard track:** PTC matches or beats JSON in 11 of 14 models.
- **Parallel fan-out:** PTC wins in 13 of 14 models—the largest margin.
- **Context rot:** JSON accuracy drops 2.3% on average; PTC accuracy is stable.
- **GPT-5.6 family:** +10.6% with PTC over JSON baseline—the largest single-model gain observed.
- **Failure cases:** PTC loses in 3/14 models on the standard track; all three are models with weaker code-generation training.

Ablations and Interpretation

Why does PTC help on parallel fan-out? PTC naturally expresses multiple calls as sequential Python statements or a list, both of which appear extensively in code pretraining. JSON parallel calls require a list of JSON objects, a format the model sees infrequently.

Why is PTC robust to context rot? Python code has clear syntactic structure (indentation, keywords, type annotations) that provides stronger anchoring than JSON when the context window is cluttered.

Why does PTC fail in 3 models? The three models where JSON wins have weaker code-generation capability, suggesting the PTC advantage is mediated by code training. For code-weak models, the structured JSON prompt may be clearer than free Python code.

Practical takeaway: For any model with strong code-generation training—which describes the majority of frontier LLMs in 2026—switching from JSON to PTC is a zero-cost accuracy gain.

Reference

Ishan Patel, Sahil Sen, Elias Lumer, Vamse Kumar Subbiah. **The Bitter Lesson of Tool Calling**. arXiv:2608.06370, August 2026. https://arxiv.org/abs/2608.06370